

ErosPulse

Text-to-Vibration Desktop Controller for the Lovense Edge 2

User & Developer Documentation — version 0.16.0

Built with Python and Tkinter. Communicates with the Lovense Remote app over the local network (Game Mode / Standard API). All third-party dependencies are bundled with the application — no installation required.

Table of Contents

1. Overview	3
Key features	3
Requirements	3
2. Getting Started	4
2.1 Running the application	4
2.2 Opening the project in Visual Studio	4
2.3 Enabling Game Mode on the phone	4
2.4 Building a standalone executable	4
3. Application Walkthrough	5
3.1 Home page	5
3.2 Connection page	5
3.3 Sequence page	5
3.4 Intensity chart	6
4. The [Y;D;[A;B]] Sequence Format	7
4.1 Example	7
4.2 Whitespace and multiple commands	7
4.3 Two motors at once, with different intensities	7
4.4 Indefinite duration (D = 0)	7
4.5 Repeating steps: LOOP(N){...}	8
5. Technical Reference	9
5.1 Local API (Lovense Game Mode)	9
5.2 Project structure	9
5.3 Design notes	10
6. Troubleshooting	11

1. Overview

ErosPulse is a Windows/Mac/Linux desktop application that converts a text-based command sequence into real-time vibration patterns on a Lovense Edge 2 device. The Edge 2 has two independent motors (an internal motor and a perineum motor), and ErosPulse can drive each one separately or together, with independent intensities and durations.

The application connects to the toy through the Lovense Remote mobile app's "Game Mode" (also called the Standard API), which exposes a local HTTP/HTTPS endpoint on the same Wi-Fi network — no developer account, API token, or cloud service is required.

Key features

- Desktop GUI built with Tkinter (no extra runtime to install).
- LAN connection to the Lovense Remote app via Game Mode — IP + port, HTTP or HTTPS.
- Independent per-motor test buttons (Motor 1 / Motor 2) on the connection screen.
- A compact, human- and machine-readable sequence format: `[Y;D;[A;B]]`, including repeat blocks (`LOOP(N){...}`).
- Real-time playback of a sequence, with Pause / Resume (physically stops the toy and resumes the current step for the exact remaining time).
- A live intensity-over-time chart for both motors, with a moving playhead during playback.
- A countdown timer (min:sec) showing total sequence duration, then remaining time during playback.
- Import ready-made sequences (.txt files) from a configurable folder, listed in a dropdown on the Sequence page.
- A ready-to-use AI prompt for turning free-form text into a valid `[Y;D;[A;B]]` sequence.
- All Python dependencies vendored inside the project folder — just run `python main.py`.
- Can be packaged into a standalone executable (.exe on Windows) via the included PyInstaller configuration.

Requirements

- Python 3.10+ installed on the computer running ErosPulse.
- Tkinter available in that Python installation (bundled with the official python.org installers on Windows and macOS; on some Linux distributions it must be installed separately, e.g. `sudo apt install python3-tk`).
- The Lovense Remote mobile app (iOS/Android), with the Edge 2 paired over Bluetooth.
- The computer and the phone connected to the same local Wi-Fi network.

2. Getting Started

2.1 Running the application

From the project folder, simply run:

```
python main.py
```

No `pip install` step is required: the `vendor/` folder inside the project already contains the required third-party packages (`requests`, `urllib3`, `certifi`, `idna`, `charset_normalizer`), and `main.py` adds that folder to the Python path automatically before importing anything else.

2.2 Opening the project in Visual Studio

The project ships with a solution file so it can be opened directly in Microsoft Visual Studio (with the Python development workload installed):

- Double-click `ErosPulse.sln` to open the solution.
- The Solution Explorer shows the full project tree (`core/`, `ui/`, `vendor/`).
- The startup file is pre-configured as `main.py`.
- If Visual Studio doesn't find a matching interpreter automatically, add one via Solution Explorer → right-click "Python Environments" → "Add Environment".

2.3 Enabling Game Mode on the phone

- 1 Open the Lovense Remote app and make sure the Edge 2 is paired over Bluetooth.
- 2 Go to **Discover** → **Game Mode**.
- 3 Enable **LAN**.
- 4 Note the local IP address and port shown on screen (HTTPS defaults to port 30010; HTTP, if enabled instead, uses port 20010).
- 5 Make sure the phone and the computer are on the same Wi-Fi network.

2.4 Building a standalone executable

The project includes a ready-to-use PyInstaller configuration (`ErosPulse.spec`) to package the app into a single standalone executable — no Python installation needed to run it afterward.

- **Windows:** double-click `build_exe.bat`, or run it from a command prompt opened in the project folder.
- **macOS / Linux:** run `./build_exe.sh` from a terminal opened in the project folder.

Either script installs PyInstaller automatically, then builds the executable using the bundled `vendor/` dependencies. The result is written to `dist/ErosPulse.exe` (Windows) or `dist/ErosPulse` (macOS/Linux).

PyInstaller does not cross-compile: build on Windows for a `.exe`, on macOS for a double-clickable `.app`, and on Linux for a native binary.

3. Application Walkthrough

3.1 Home page

The home page shows the ErosPulse brand, a short description, a live connection status indicator, and two buttons:

- **Start** — goes to the Connection page if no toy is connected yet, or directly to the Sequence page if a toy is already connected.
- **Configure connection** — goes to the Connection page.

3.2 Connection page

Fields and controls:

- **Local IP** and **Port** — the values shown in Game Mode on the phone.
- **Use HTTPS** checkbox — must match the HTTPS setting shown in Game Mode.
- **Connect** — verifies the connection and lists the detected toy(s), without triggering any vibration.
- **Test Motor 1 (internal) / Test Motor 2 (perineum)** — each button connects automatically if not already connected, then sends a short vibration pulse to that specific motor only, to confirm wiring and motor identification.

Motor 1 and Motor 2 are addressed independently using the Lovense protocol actions "Vibrate1" and "Vibrate2".

3.3 Sequence page

This is where a [Y;D;[A;B]] sequence (see chapter 4) is pasted, imported, previewed, and played back on the connected toy.

- A hint label reminds the format, and reflows automatically when the window is resized.
- A **template dropdown** lists the .txt files found in the configured import folder (Import / at the project root by default). Selecting one loads its content into the editor. A refresh button re-scans the folder, and a "Folder..." button lets you pick a different folder (remembered between launches) via the system's native folder picker.
- A **Show/Hide text** button collapses or expands the editor; its content is preserved either way.
- The text box accepts up to 4000 characters.
- A live **intensity chart** (see 3.4) previews both motors' timelines as you type or import a template.
- A **duration counter** (min:sec) shows the total length of the parsed sequence; during playback it turns into a live countdown of the time remaining.
- A character counter is always visible in the status bar at the bottom of the window.
- **Generate the vibration pattern** — parses the text and starts playback on the connected toy. If no toy is connected, the app redirects to the Connection page instead.
- **Pause / Resume** — pausing immediately stops the toy physically; resuming re-sends the current step for exactly the time that was left when it was paused (not from the beginning).
- A progress line shows which step is currently playing, e.g. Step 2/5 — [2;1;[0;12]]. This information is kept and restored across pause / navigate-away / resume, instead of being replaced by a generic "Resuming..." message.

Leaving the page (Home button) while a sequence is playing automatically pauses it — it does not stop it. Coming back to the Sequence page and pressing Resume continues exactly where playback left off, even after visiting other pages.

3.4 Intensity chart

A small step-chart at the bottom of the Sequence page shows, for each motor, its intensity (0–20) plotted against time. Motor 1 and Motor 2 are drawn in different colors with a legend; a dashed vertical line ("playhead") tracks the current playback position in real time, freezing while paused. A motor not addressed by the currently displayed step is drawn at 0, matching the toy's actual behavior (a single-motor command stops whatever the other motor was doing, unless explicitly combined — see 4.3). The chart is drawn directly on a Tkinter Canvas; no external charting library is used.

4. The [Y;D;[A;B]] Sequence Format

ErosPulse drives the toy from a compact command format, designed to be easy for both a person and an AI language model to read and generate:

```
[Y;D;[A;B]]
```

Field	Meaning	Valid values
Y	Which motor(s) to activate	1 = Motor 1 (internal) only 2 = Motor 2 (perineum) only 3 = both motors together
D	Duration of the step, in seconds	Integer ≥ 0 . 0 means an indefinite loop on the toy side, and ends the sequence right there (see 4.4).
A	Motor 1 intensity	Integer 0–20 (out-of-range values are automatically clamped)
B	Motor 2 intensity	Integer 0–20 (out-of-range values are automatically clamped)

4.1 Example

```
[1;2;[10;0]][2;3;[0;15]][3;4;[8;12]]
```

This plays, back to back:

- Motor 1 only, for 2 seconds, at intensity 10 (Motor 2 unused $\rightarrow$ 0).
- Motor 2 only, for 3 seconds, at intensity 15 (Motor 1 unused $\rightarrow$ 0).
- Both motors together, for 4 seconds, Motor 1 at intensity 8 and Motor 2 at intensity 12.

Total duration: 9 seconds.

4.2 Whitespace and multiple commands

Extra spaces around the separators are tolerated when parsing (e.g. [2 ; 3 ; [0 ; 15]] is valid). Multiple commands can be concatenated directly, with or without line breaks between them — the parser extracts every valid [Y;D;[A;B]] pattern it finds in the text, in order.

4.3 Two motors at once, with different intensities

When Y=3, ErosPulse sends two separate network requests: one for each motor. The Lovense API's stopPrevious flag is used on the second request so that Motor 2 starting does not cancel Motor 1, allowing both motors to run simultaneously at different intensities.

4.4 Indefinite duration (D = 0)

A duration of 0 means the toy vibrates indefinitely once that step starts — there is no way to know when it will stop. For this reason, once ErosPulse plays a step with D=0, it does not attempt to play any further steps

after it; the sequence effectively ends there. The intensity chart reflects this the same way, with a note next to the truncated timeline.

4.5 Repeating steps: LOOP(N){...}

To repeat a group of steps without retyping it, wrap it in a `LOOP(N){ . . . }` block:

```
LOOP(N){ Z }
```

N is the number of repetitions (integer ≥ 0 ; N=0 makes the block a no-op), and Z is one or more `[Y;D;[A;B]]` steps. For example:

```
LOOP(3){[1;2;[10;0]] [2;2;[0;15]]}
```

plays that pair of steps three times in a row (6 steps total). Loop blocks can be mixed with ordinary steps before and after them, and can be nested (`LOOP(2){ . . . LOOP(3){ . . . } }`). Parsing fully expands every loop into a flat list of steps before playback, so the rest of the app (playback, the intensity chart, the duration counter) never needs to know loops exist — they only ever see the final, expanded sequence. An unclosed `LOOP( . . . )` block (a missing closing brace) is reported as a clear error rather than silently ignored.

5. Technical Reference

5.1 Local API (Lovense Game Mode)

ErosPulse communicates with the Lovense Remote app's local HTTP(S) API, sending JSON requests to `{scheme}://{host}:{port}/command`. Supported commands used by the app:

Command	Purpose
GetToys	Lists connected toys and their capabilities.
Function	Vibrate / Vibrate1 / Vibrate2 / Stop — direct motor control.
Pattern	Sends a custom intensity sequence at a fixed interval (available in the client, not yet exposed in the UI).
Preset	Plays one of four built-in patterns: pulse, wave, fireworks, earthquake (available in the client, not yet exposed in the UI).

HTTPS uses a self-signed certificate on the local network, so certificate verification is disabled for these local calls only — the same approach used by Lovense's own official SDKs.

5.2 Project structure

ErosPulse.sln / ErosPulse.pyproj	Visual Studio project files
ErosPulse.spec	PyInstaller build configuration
build_exe.bat / build_exe.sh	One-step executable build scripts
main.py	Entry point; adds vendor/ to sys.path
requirements.txt	Reference list of dependencies (optional)
core/	
constants.py	Shared limits (min/max strength, pattern size)
lovense_client.py	Network client for the local Lovense API
vibration_command.py	[Y;D;[A;B]] / LOOP(N){...} parsing, playback
pattern_generator.py	Programmatic waveform pattern generator
ai_prompt.py	System prompt for AI-based sequence generation
settings.py	Persisted settings (e.g. import folder)
template_library.py	Discovers/reads .txt sequence templates
app_state.py	Shared connection state across pages
version.py	Centralized app name and version number
ui/	
theme.py	Shared colors and fonts
app_window.py	Main window, lazy page loading/navigation
pages/	
home_page.py	Landing page
connection_page.py	Connection setup and per-motor test buttons
text_page.py	Sequence editor, import, playback, chart
widgets/	
sequence_chart.py	Canvas-based intensity-over-time chart
vendor/	Bundled third-party dependencies
Import/	Default folder scanned for .txt templates

5.3 Design notes

- Each file has a single, clearly-named responsibility (one feature per file), to keep the codebase easy to navigate and extend.
- Pages are loaded lazily: only the Home page (which has zero external dependencies) is built at startup, so a missing dependency on another page cannot prevent the app from opening at all.
- All motor intensities are clamped to the 0–20 range at the data-model level (`VibrationCommand`), so invalid values from any source — manual input, an imported template, or AI-generated — can never reach the toy.
- Playback runs in a background thread so the Tkinter UI never freezes during a multi-second (or multi-minute) sequence.
- Persisted settings (currently: the import folder) live in `settings.py`, kept separate from the in-memory-only connection state in `app_state.py` — one survives between launches, the other doesn't.

6. Troubleshooting

Symptom	Likely cause / fix
Connection fails / times out	Verify Game Mode + LAN are enabled on the phone, the IP/port match what's shown there, the HTTPS checkbox matches the app's setting, and both devices are on the same Wi-Fi network.
Connected, but no toy detected	Make sure the Edge 2 is paired over Bluetooth inside the Lovense Remote app first.
"Missing dependency" error when opening a page	Run the app with the bundled vendor/ folder present (default), or install requirements.txt if running the source directly outside the provided project structure.
A step in my sequence never seems to end	Its duration D is 0 (indefinite). See section 4.4 — this intentionally stops the sequence there.
Window text/buttons get cut off when resized	Resize the window to at least its minimum size; the layout is anchored so the countdown, character count and controls stay visible, with only the text area growing or shrinking.
My template doesn't show up in the dropdown	Confirm it's a .txt file inside the configured import folder (shown next to the folder picker), then click the refresh button. The list also refreshes automatically each time you return to the Sequence page.
build_exe.bat / build_exe.sh fails	Make sure Python is installed and available on PATH, and that you're running the script on the OS you want an executable for (PyInstaller does not cross-compile).